E22 — Algoritmos de ordenamiento: Inserción y Selección

🎯 Objetivo del ejercicio

- Comprender el funcionamiento del algoritmo de ordenamiento por inserción (Insertion Sort).
- Comprender el funcionamiento del algoritmo de ordenamiento por selección (Selection Sort).
- Reflexionar sobre las diferencias en eficiencia, número de comparaciones y movimientos.

📘 Teoría y conceptos

1. Insertion Sort

- Recorre la lista elemento por elemento.
- Cada elemento se "inserta" en la posición correcta dentro de la parte ya ordenada.
- Funciona muy bien con listas casi ordenadas.
- Es intuitivo: simula cómo ordenarías cartas en tu mano.

2. Selection Sort

- Recorre la lista buscando el **mínimo** en cada pasada.
- Intercambia el mínimo encontrado con la posición actual.
- Hace siempre el mismo número de comparaciones, aunque la lista esté ordenada.
- Es fácil de entender, pero no muy eficiente.

📝 Instrucciones

1. Copiar cada código en Python Tutor

- Ejecutar paso a paso (Next → Next → Next).
- Observar comparaciones, intercambios y movimientos.

2. Probar ambos algoritmos con las mismas listas

Sugerencias de listas para comparar:

1. **Lista ya ordenada:** [1, 2, 3, 4, 5]
2. **Lista en orden inverso:** [5, 4, 3, 2, 1]
3. **Lista casi ordenada (solo un elemento fuera de lugar):** [1, 2, 4, 3, 5]

🧩 *Código 1 — Insertion Sort (Ordenamiento por Inserción)*

```
lista = [5, 4, 3, 2, 1]   # Cambia esta lista por las sugeridas
print("Lista original:", lista)
for i in range(1, len(lista)):
    actual = lista[i]
    j = i - 1
    print("\nInsertando:", actual)
    while j >= 0 and lista[j] > actual:
        print("  ", lista[j], "es mayor que", actual, "-> mover a la derecha")
        lista[j + 1] = lista[j]
        j -= 1
    lista[j + 1] = actual
    print("  Lista después de insertar:", lista)
print("\nLista ordenada:", lista)
```

Código 2 — Selection Sort (Ordenamiento por Selección)

```
lista = [5, 4, 3, 2, 1]   # Cambia esta lista por las sugeridas
print("Lista original:", lista)
n = len(lista)
for i in range(n):
    min_index = i
    print("\nBuscando mínimo desde la posición", i)
    for j in range(i + 1, n):
        print("  Comparando", lista[j], "con", lista[min_index])
        if lista[j] < lista[min_index]:
            print("   -> Nuevo mínimo encontrado:", lista[j])
            min_index = j
    if min_index != i:
        print("  Intercambiando", lista[i], "con", lista[min_index])
        lista[i], lista[min_index] = lista[min_index], lista[i]
    print("  Lista después de la pasada:", lista)
print("\nLista ordenada:", lista)
```

Entregable

Cada equipo entregará **una hoja escrita a mano o una redacción en Word** con:

1. Explicación del funcionamiento

- ¿Cómo funciona Insertion Sort?
- ¿Cómo funciona Selection Sort?

2. Comparación entre ambos

- ¿Cuál hace más comparaciones?
- ¿Cuál hace más movimientos?
- ¿Cuál es más rápido en listas casi ordenadas?
- ¿Cuál se comporta igual sin importar el orden inicial?

3. Observaciones de Python Tutor

- ¿Qué vieron paso a paso?
- ¿Qué les llamó la atención?
- ¿Qué algoritmo les pareció más intuitivo?

4. Conclusión

- ¿Cuál algoritmo usarían en un caso real y por qué?